

ReviveAI - Safe Automated Revenue Recovery

ReviveAI is a safe automated revenue recovery platform designed to evaluate failed payment events and recommend appropriate recovery actions. The system combines a risk assessment component, failure diagnosis, a deterministic policy engine, an action engine, and a verification engine. Its central design principle is that AI-assisted recommendations do not directly control payment execution. Instead, deterministic policy guardrails decide whether an action is permitted. The current implementation operates in simulation mode, allowing recovery scenarios to be tested without interacting with real payment gateways or moving real money.

Problem Statement

A payment recovery system must answer three important questions: whether a failed payment is likely to be recoverable, which recovery strategy is appropriate, and whether the proposed action is safe to execute. A system that focuses only on recovery probability may retry unsuitable transactions, exceed configured limits, or create repeated customer charges. ReviveAI separates prediction from execution to reduce these risks.

Proposed Solution

ReviveAI processes each payment event through a sequential recovery pipeline:

Stage	Purpose
Payment Event	Receives transaction and failure information.
Risk Detector	Estimates recovery probability and assigns a risk level.
Diagnosis Engine	Classifies the failure and identifies whether it is retryable.
Policy Engine	Applies deterministic safety rules and guardrails.
Action Engine	Simulates the approved recovery action.
Verification Engine	Checks the simulated result and records the outcome.

Implementation Details

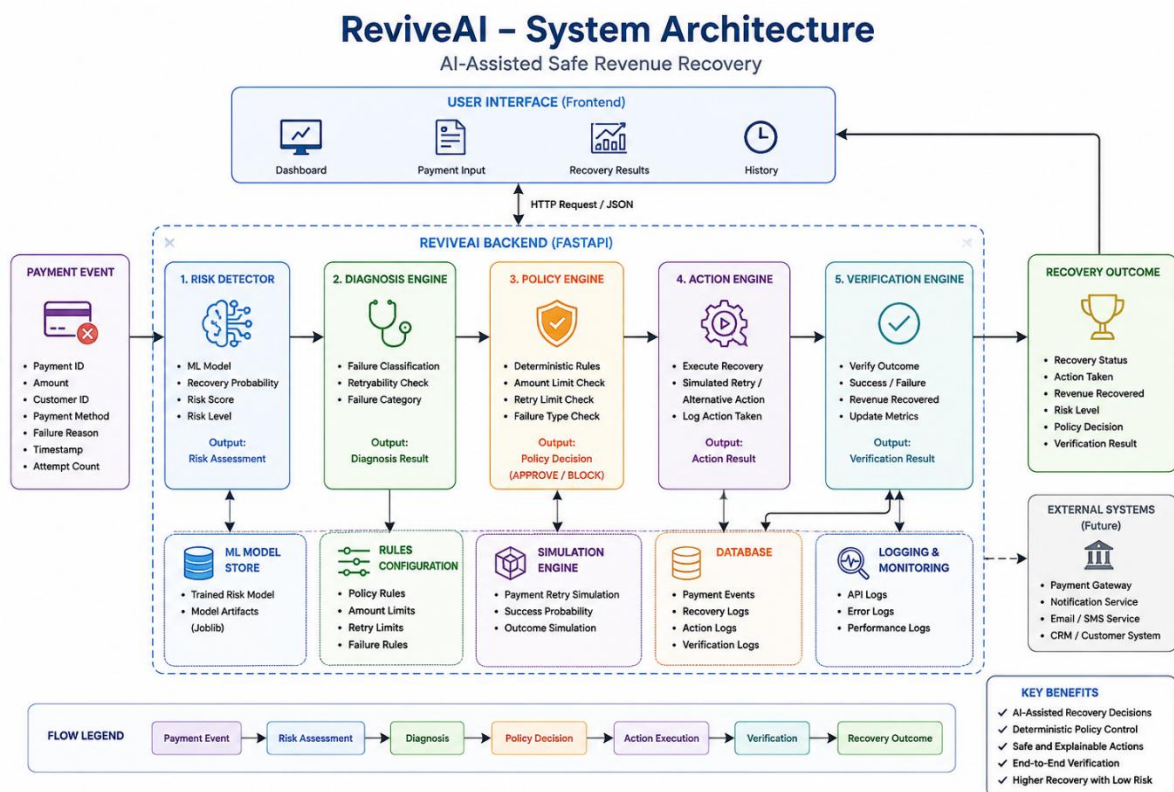
Component	Technology / Description
Backend	FastAPI-based REST API
Frontend	HTML, CSS, and JavaScript dashboard
Containerization	Docker
Recovery endpoint	POST /api/v1/recover

Execution mode	Simulation mode
Decision design	AI-assisted assessment with deterministic policy control

System Architecture

The application uses a modular architecture. The frontend provides the recovery control center where users enter payment details and view the decision. The FastAPI backend exposes the recovery endpoint and coordinates the processing pipeline. The engines are separated into logical stages so that risk estimation, diagnosis, policy evaluation, action execution, and verification can be tested independently.

The application is containerized using Docker. This makes the runtime environment more consistent and simplifies installation for evaluators who clone the GitHub repository.



Implementation Details

Component	Technology / Description
Backend	FastAPI-based REST API
Frontend	HTML, CSS, and JavaScript dashboard
Containerization	Docker
Recovery endpoint	POST /api/v1/recover

Execution mode	Simulation mode
Decision design	AI-assisted assessment with deterministic policy control

Recovery Decision Logic

The system considers transaction amount, payment method, customer type, failure type, failure code, attempt count, previous transaction history, and previous success rate. These inputs support the risk and diagnosis stages. The policy engine then checks whether the proposed action satisfies configured limits and safety conditions.

For a transient failure such as a bank timeout or network error, the system may recommend a retry. For a hard failure such as insufficient funds or a permanently declined card, the system can block automatic recovery and require customer action. The policy engine can also block an otherwise promising transaction when it exceeds the automatic recovery amount limit or another safety rule.

Safety and Reliability Features

- Separation of recommendation and execution through a deterministic policy engine.
- Automatic recovery limits to prevent unsafe high-value actions.
- Retryability classification based on the failure type and failure code.
- Simulation mode to avoid real payment execution during testing.
- Verification stage to record whether the simulated action succeeded.
- Structured outputs that make decisions explainable and easier to audit.
- Modular pipeline design that supports independent testing of each engine.

Testing and Demonstration

The dashboard was tested using payment-event scenarios containing transaction details, failure information, customer history, and retry context. The output displays the recovery probability, risk level, diagnosis, policy decision, action, and verification status. In an approved simulation scenario, the system produced a recovery probability of 87.5%, classified the event as HIGH risk, selected `RETRY_PAYMENT`, approved the action, and reported a simulated recovery of ₹5,000.00. In a blocked scenario, the system produced a recovery probability of 73.6%, selected `RETRY_PAYMENT`, but blocked execution because the transaction exceeded the automatic recovery amount limit. Verification was skipped and the recovered amount was ₹0.00. These scenarios demonstrate that a high recovery probability does not override safety policies.

Batch Simulation Results

- To validate the system beyond individual scenarios, ReviveAI was evaluated on a batch simulation of 100 payment transactions. All 100 transactions were processed successfully with zero execution failures.

Metric	Result
Total transactions processed	100
Execution failures	0
Transactions recovered	36
Transactions blocked by policy engine	26
Transactions not recovered	38
Total revenue at risk	₹383,848.15
Total revenue recovered	₹168,208.58
Revenue not recovered	₹215,639.57
Revenue recovery rate	43.82%
Transaction recovery rate	36.00%

- The 43.82% revenue recovery rate represents the proportion of revenue recovered from the total revenue at risk. It is different from the 36.00% transaction recovery rate, which represents the proportion of transactions recovered.
- The simulation also demonstrated that policy approval and financial recovery are separate outcomes. For example, TX0000100 received an approved payment-method-update action, but verification confirmed that the payment was not recovered. Similarly, TX0000098 was escalated after reaching three attempts rather than being automatically retried again.

- These results demonstrate that ReviveAI can process payment events through its complete recovery pipeline while applying deterministic safety controls and recording verified outcomes.

Limitations

- The current implementation is a simulation and does not connect to a live payment gateway.
- Recovery probabilities depend on the implemented risk logic and are not yet validated against a large production dataset.
- The system does not currently perform real settlement, refund, or payment-status reconciliation.
- Production deployment would require authentication, monitoring, secure secret management, and stronger persistence.

Future Scope

- Integrate with a payment gateway sandbox for end-to-end payment-status testing.
- Train and validate a recovery-risk model using historical payment outcomes.
- Add persistent transaction records and audit logs.
- Introduce configurable policy rules through an administrative interface.
- Add monitoring, alerts, authentication, and role-based access control.
- Evaluate recovery performance using precision, recall, recovery rate, and false-retry rate.

Conclusion

ReviveAI demonstrates a controlled approach to automated payment recovery. Instead of allowing an AI recommendation to directly trigger a payment action, the system places deterministic policy guardrails between prediction and execution. This architecture improves safety, explainability, and testability while allowing recovery scenarios to be evaluated through a practical dashboard. The Dockerized implementation also supports reproducible setup and evaluation from the project repository.

Repository and Installation

The complete source code, Docker configuration, frontend, backend, and README should be provided in the GitHub repository. Evaluators can clone the repository, build the Docker image, run the container, and open the frontend to test the recovery workflow. The application should be clearly marked as simulation mode.

Commands:

```
docker build -t reviveai .
```

```
docker run -d -p 8000:8000 --name reviveai-container reviveai
```